# Compte Rendu — Atelier Pratique MLOps

Intégration de Git + DVC + MLflow dans un Workflow Reproductible (Classification Iris & Wine)

| | |
|---|---|
| **Étudiant :** Mohamed Amine BELGUEROUANI | **Date :** 02 Novembre 2026 |
| **Matière :** MLOps & Industrialisation ML | **Datasets :** Iris (150 lignes) & Wine (178 lignes) |
| **Outils :** Python 3.12, Git 2.52, DVC 3.67, MLflow 3.17 | **Backend MLflow :** SQLite (sqlite:///mlflow.db) |
| **Repository GitHub :** https://github.com/anakin-skyguy/atelier-mlops | **Validation :** Pytest (4/4 passés) + Clone isolé HTTPS |

## 1. Architecture Générale et Complémentarité Git + DVC + MLflow

Contrairement au développement logiciel classique où seul le code détermine le comportement du programme, un modèle de Machine Learning dépend conjointement du **code source**, des **jeux de données** et des **hyperparamètres**. Cet atelier intègre trois briques complémentaires pour garantir traçabilité et reproductibilité :

| Fonctionnalité MLOps | Git / GitHub | DVC | MLflow |
|---|---|---|---|
| **Versioning du code source** | Oui (commits, branches) | Non | Non (trace le SHA) |
| **Versioning des datasets** | Pointeurs .dvc | Oui (cache + remote) | Métadonnées |
| **Pipeline ML reproductible** | dvc.yaml & dvc.lock | Oui (dvc repro / dag) | Tracking associé |
| **Suivi des hyperparamètres** | params.yaml | dvc params diff | Oui (log_params) |
| **Suivi des métriques** | reports/metrics*.json | dvc metrics show/diff | Oui (log_metrics) |
| **Suivi des artefacts modèles** | Exclus (.gitignore) | Oui (sorties outs:) | Oui (log_artifact) |
| **Comparaison visuelle des runs** | Non | Comparaison CLI | Oui (UI & graphiques) |

## 2. TP 1 — Gestion du Code Source avec Git et Publication GitHub

Le dépôt Git a été structuré autour des branches main (versions stables publiées) et develop (intégration progressive), jalonné par **14 commits atomiques** et trois tags sémantiques annotés : v1.0 (Random Forest 100 arbres), v2.0 (Random Forest 200 arbres) et v3.0 (pipeline complet RF + KNN + tests + CI).

- **Configuration .gitignore & .gitattributes :** Exclusion de mlruns/, mlflow.db, .venv/ et __pycache__/ tout en déléguant models/.gitignore à DVC. L'ajout de * text=auto eol=lf dans .gitattributes garantit des empreintes MD5 identiques entre Windows et Linux.
- **Réponse à la question préliminaire (p. 3) :** Stocker directement des datasets volumineux dans Git duplique chaque révision dans .git/objects, ralentit fortement git clone, dépasse la limite de 100 Mo/fichier sur GitHub et produit des diffs illisibles.

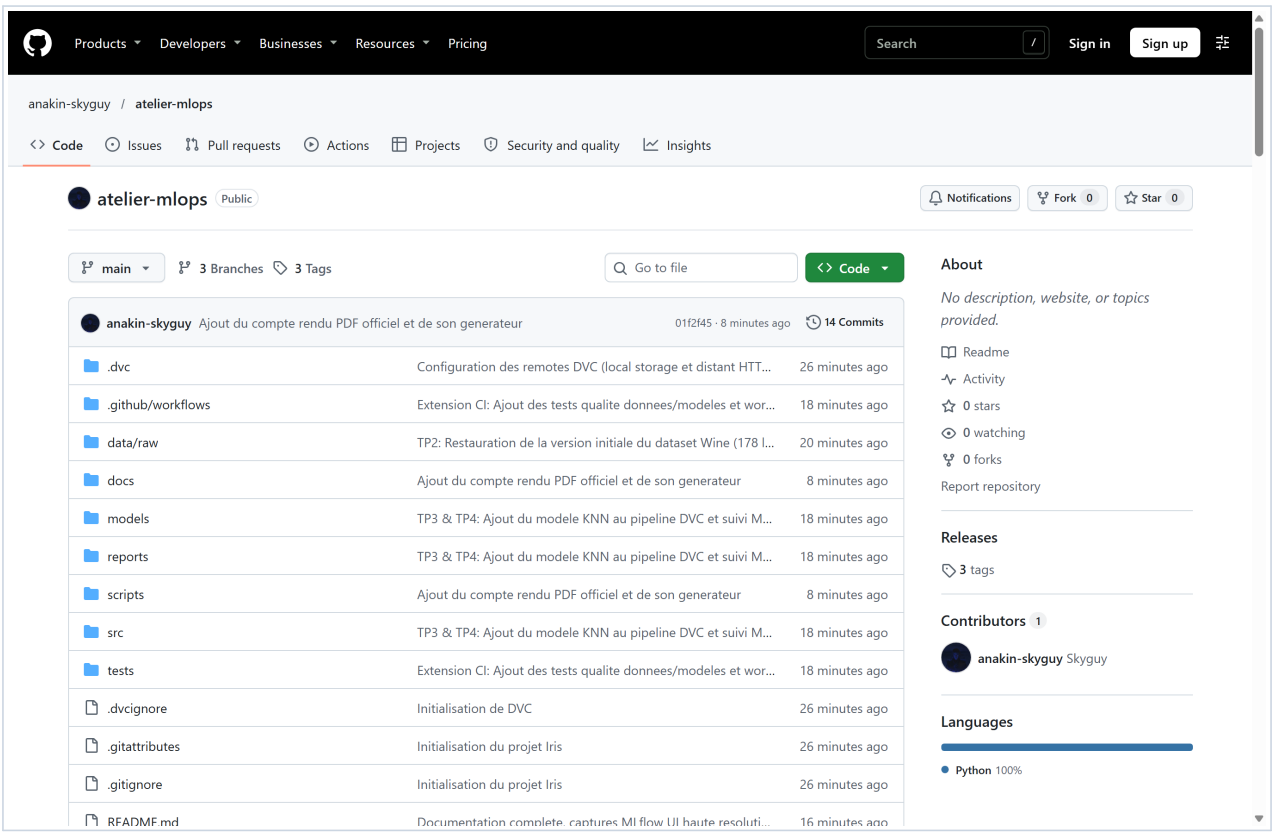



*Figure 1 — Dépôt GitHub publié (https://github.com/anakin-skyguy/atelier-mlops) : branches main/develop, 3 tags (v1.0, v2.0, v3.0) et arborescence MLOps sans fichiers binaires ni CSV.*

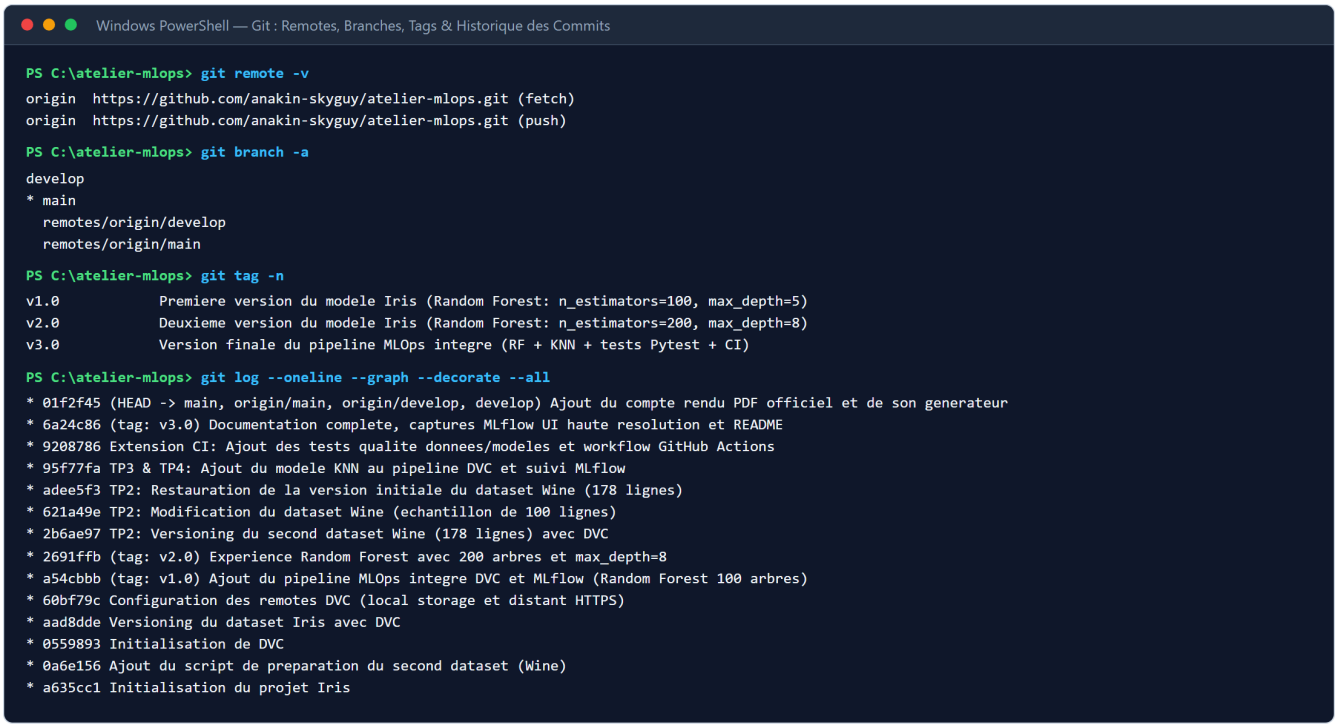



*Figure 2 — Vérification CLI Git : dépôt distant GitHub, branches locales/distantes, tags annotés (v1.0, v2.0, v3.0) et graphe complet des commits.*

## 3. TP 2 — Versioning des Données avec DVC, Remotes et Restauration

Conformément au TP 2, deux datasets distincts ont été versionnés avec dvc add : le dataset principal **Iris** (data/raw/iris.csv, 150 lignes) et le second dataset **Wine** (data/raw/wine.csv, 178 lignes). Deux remotes sont configurés dans .dvc/config : storage (../dvc-storage local) et github (remote HTTPS par défaut adossé à la branche orpheline dédiée dvc-storage permettant un dvc pull immédiat sur toute machine externe).

**Expérience de modification et restauration (TP 2 — Q7) :** Le dataset wine.csv a été réduit de 178 à 100 lignes (MD5 modifié de 2925c4a8... vers cc50fa13...), puis restauré via git checkout HEAD~1 -- data/raw/wine.csv.dvc && dvc checkout, rétablissant exactement les 178 lignes et le MD5 d'origine.

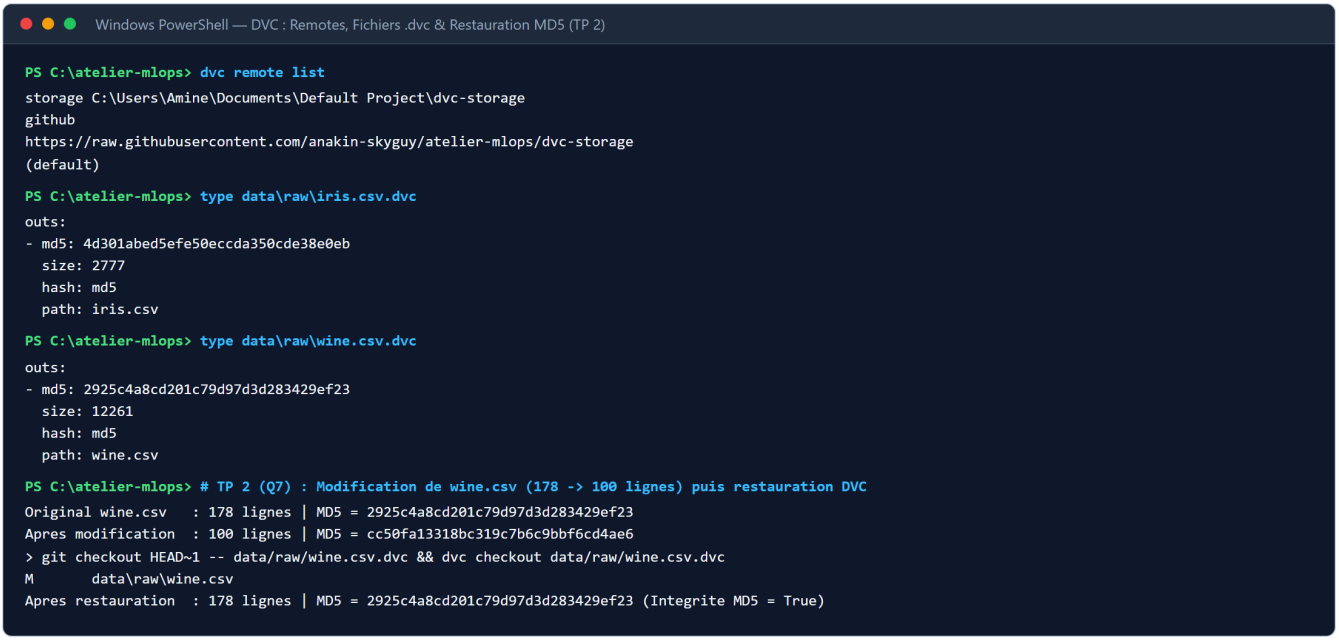



*Figure 3 — Versioning DVC des datasets Iris et Wine : configuration des remotes, contenu des pointeurs .dvc et vérification par hachage MD5 de la restauration de wine.csv (178 → 100 → 178 lignes).*

## 4. TP 3 — Suivi et Comparaison des Expérimentations dans MLflow UI

Le serveur de suivi MLflow s'appuie sur le backend relationnel SQLite (sqlite:///mlflow.db) au sein de l'expérience TP_Iris_Classification. Trois expérimentations ont été conduites sur le même découpage stratifié (test_size=0.2 soit 120 observations d'entraînement et 30 de test, random_state=42) :

| Tag Git | Nom du Run MLflow | Modèle & Hyperparamètres | Accuracy | Precision | Recall | F1-Score |
|---|---|---|---|---|---|---|
| **v1.0** | RandomForest_n100_d5 | Random Forest (n=100, max_depth=5) | **0.9333** | **0.9333** | **0.9333** | **0.9333** |
| **v2.0** | RandomForest_n200_d8 | Random Forest (n=200, max_depth=8) | 0.9000 | 0.9024 | 0.9000 | 0.8997 |
| **v3.0** | KNN_k5 | K-Nearest Neighbors (n_neighbors=5) | **1.0000** | **1.0000** | **1.0000** | **1.0000** |

**Analyse comparative :** L'augmentation de complexité du Random Forest entre v1.0 (100 arbres, profondeur 5) et v2.0 (200 arbres, profondeur 8) réduit l'accuracy de 0.9333 (28/30) à 0.9000 (27/30), illustrant un léger sur-ajustement aux frontières locales du jeu d'apprentissage. Le modèle KNN_k5 atteint 1.0000 (30/30) sur ce pli de test grâce à la forte séparabilité géométrique des pétales/sépales ; compte tenu de la taille modeste du test (30 individus, soit 3,33 % par erreur), une validation croisée stratifiée serait requise en production pour confirmer cet avantage.

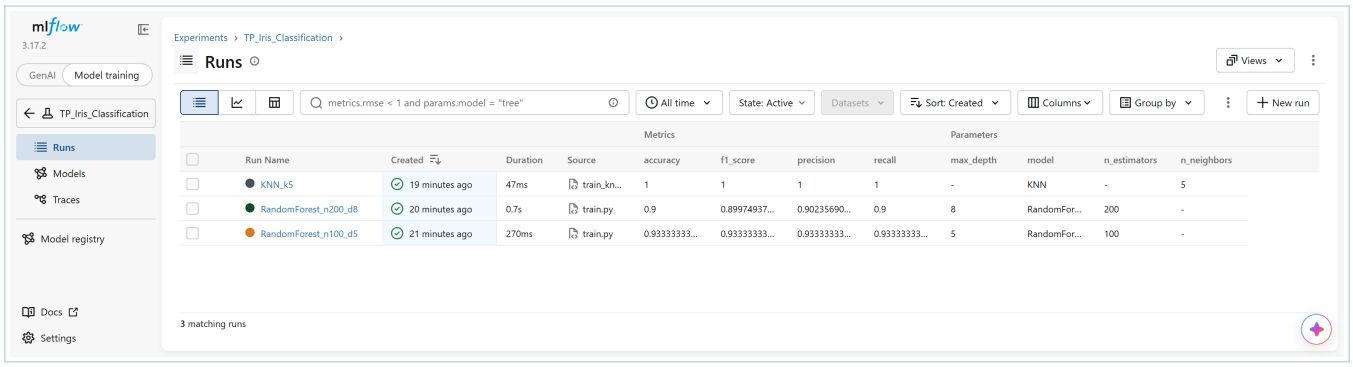



*Figure 4 — Interface MLflow UI (mode Model training) : liste des 3 runs de TP_Iris_Classification affichant l'ensemble des métriques (accuracy, f1_score, precision, recall) et des hyperparamètres.*

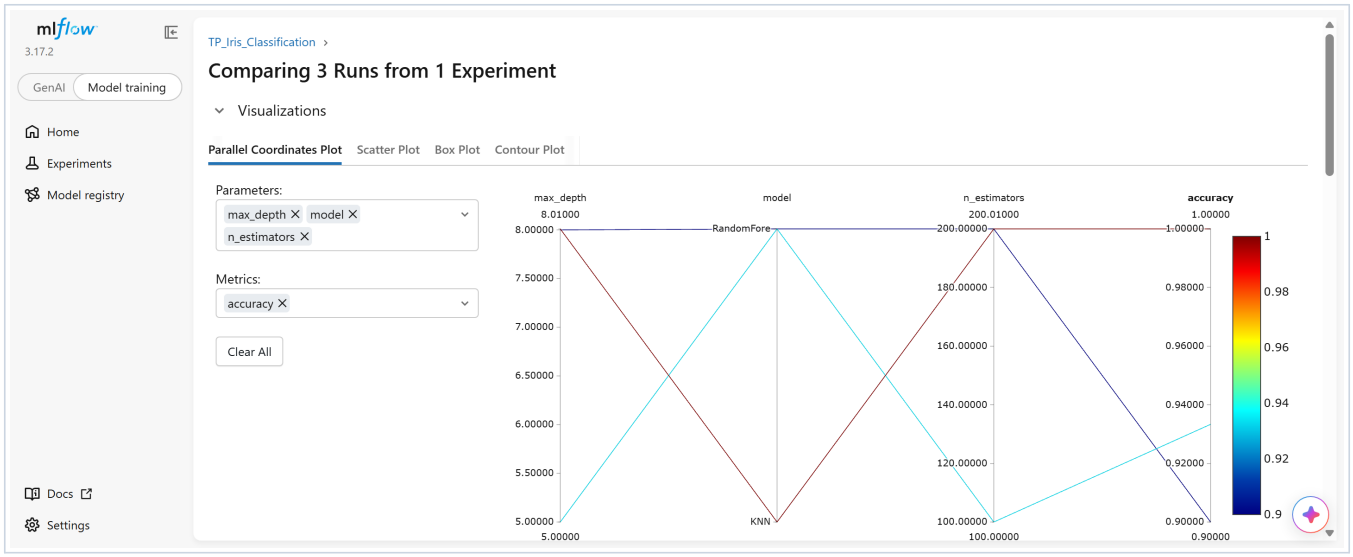
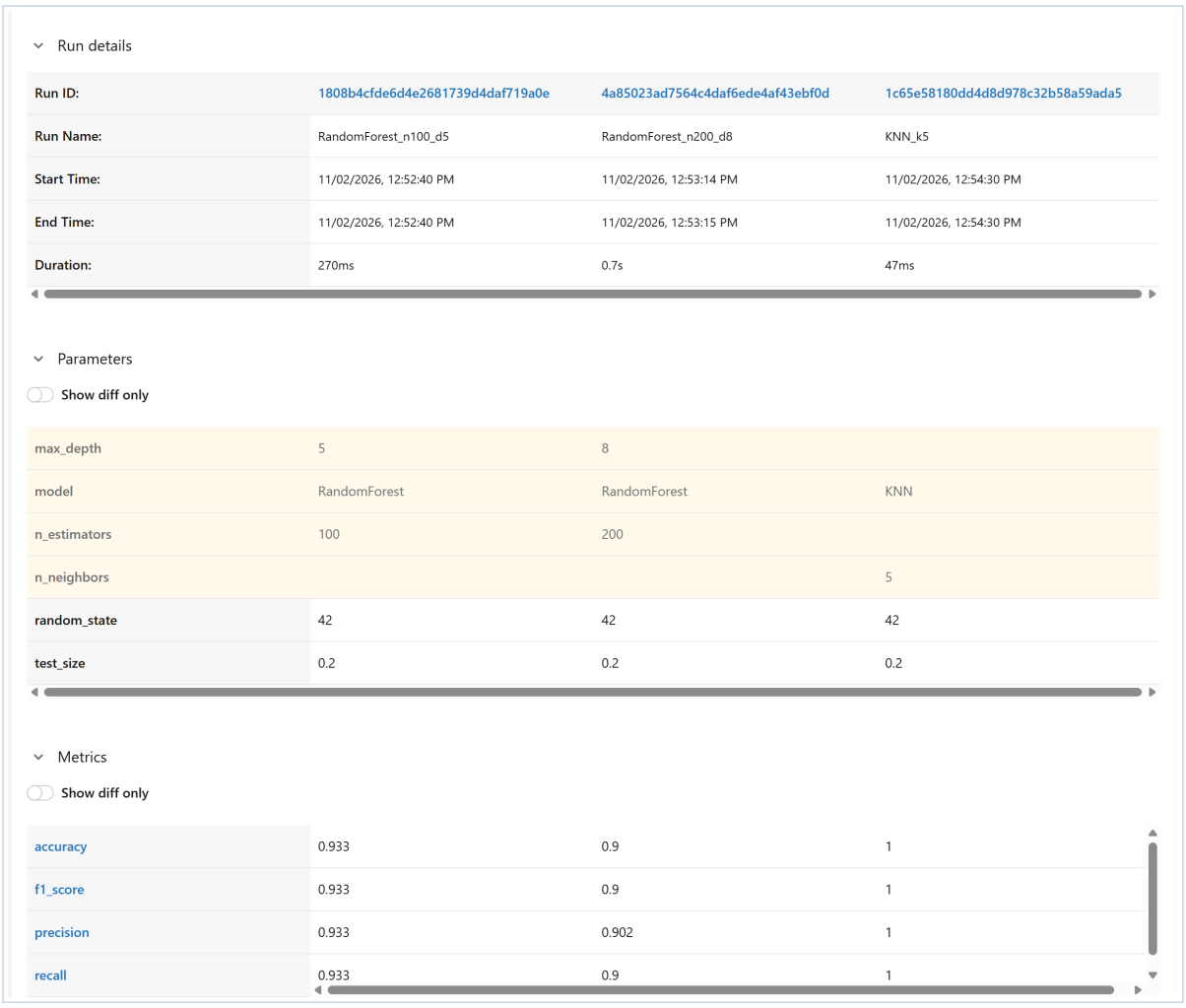



*Figure 5 — Comparaison visuelle des 3 runs dans MLflow UI : graphique en coordonnées parallèles reliant max_depth, type de modèle, n_estimators et accuracy.*



Run details

| Run ID: | 1808b4cfde6d4e2681739d4daf719a0e | 4a85023ad7564c4daf6ede4af43ebf0d | 1c65e58180dd4d8d978c32b58a59ada5 |
|---|---|---|---|
| Run Name: | RandomForest_n100_d5 | RandomForest_n200_d8 | KNN_k5 |
| Start Time: | 11/02/2026, 12:52:40 PM | 11/02/2026, 12:53:14 PM | 11/02/2026, 12:54:30 PM |
| End Time: | 11/02/2026, 12:52:40 PM | 11/02/2026, 12:53:15 PM | 11/02/2026, 12:54:30 PM |
| Duration: | 270ms | 0.7s | 47ms |

Parameters

Show diff only

| max_depth | 5 | 8 | |
|---|---|---|---|
| model | RandomForest | RandomForest | KNN |
| n_estimators | 100 | 200 | |
| n_neighbors | | | 5 |
| random_state | 42 | 42 | 42 |
| test_size | 0.2 | 0.2 | 0.2 |

Metrics

Show diff only

| accuracy | 0.933 | 0.9 | 1 |
|---|---|---|---|
| f1_score | 0.933 | 0.9 | 1 |
| precision | 0.933 | 0.902 | 1 |
| recall | 0.933 | 0.9 | 1 |

*Figure 6 — Tableau comparatif côte à côte dans MLflow UI : détails d'exécution, différences d'hyperparamètres surlignées et métriques complètes des 3 modèles.*

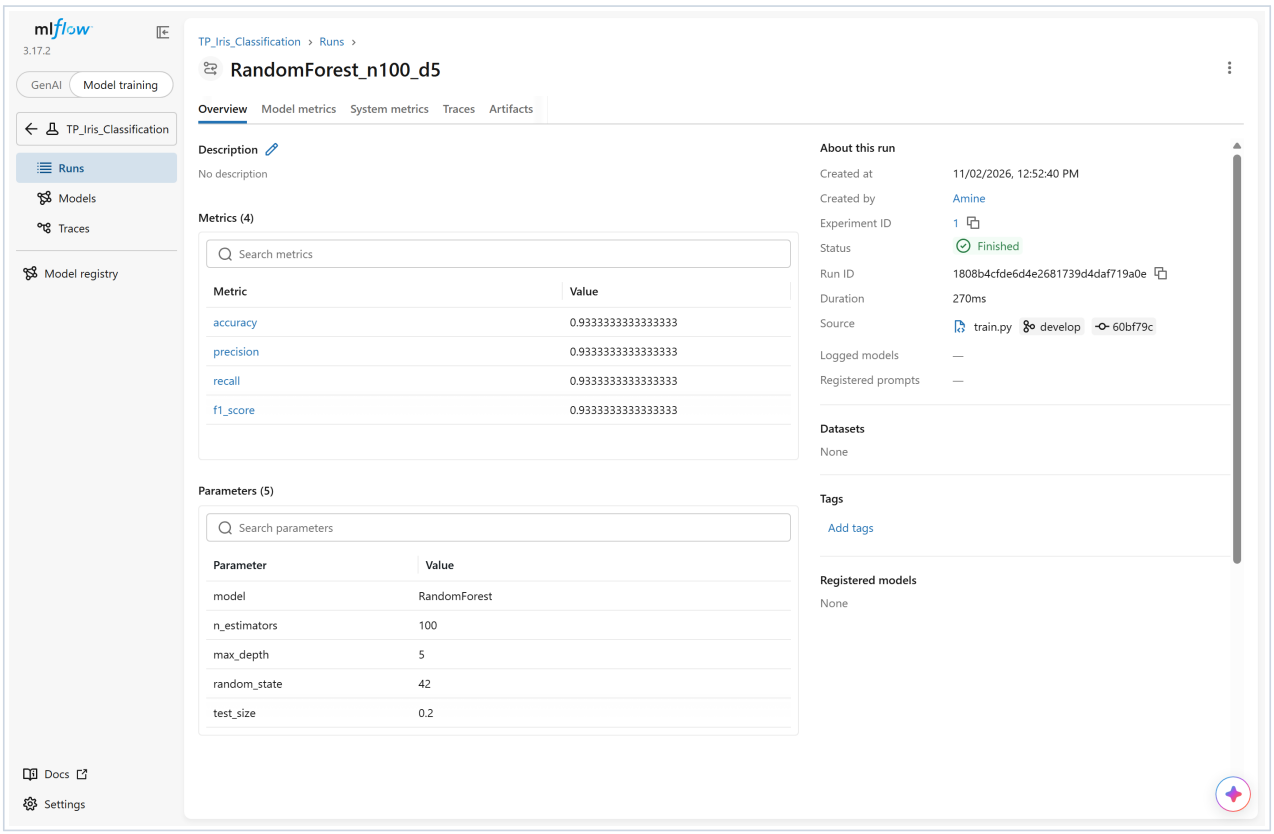



*Figure 7 — Fiche détaillée du run RandomForest_n100_d5 (tag v1.0 : Accuracy = 0.9333, n_estimators = 100, max_depth = 5).*

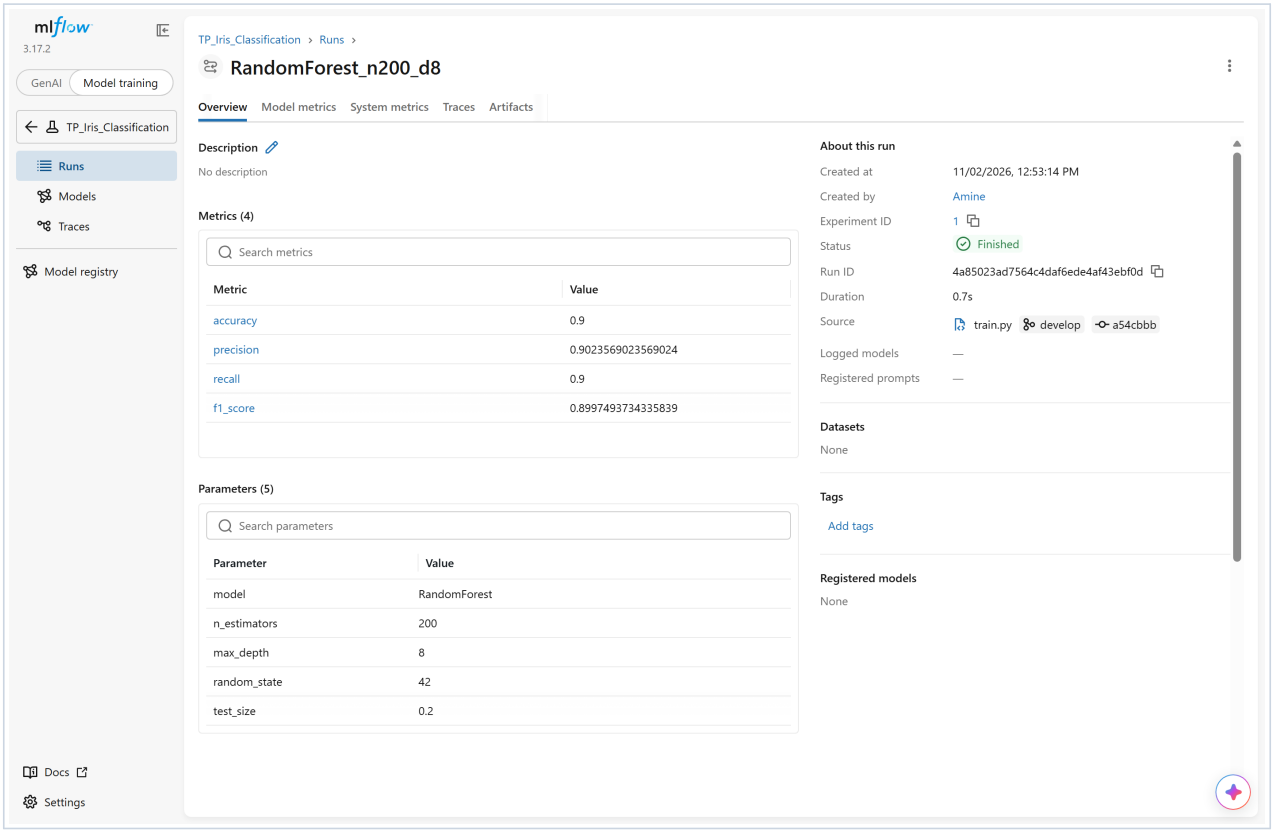



*Figure 8 — Fiche détaillée du run RandomForest_n200_d8 (tag v2.0 : Accuracy = 0.9000, F1 = 0.8997, n_estimators = 200, max_depth = 8).*

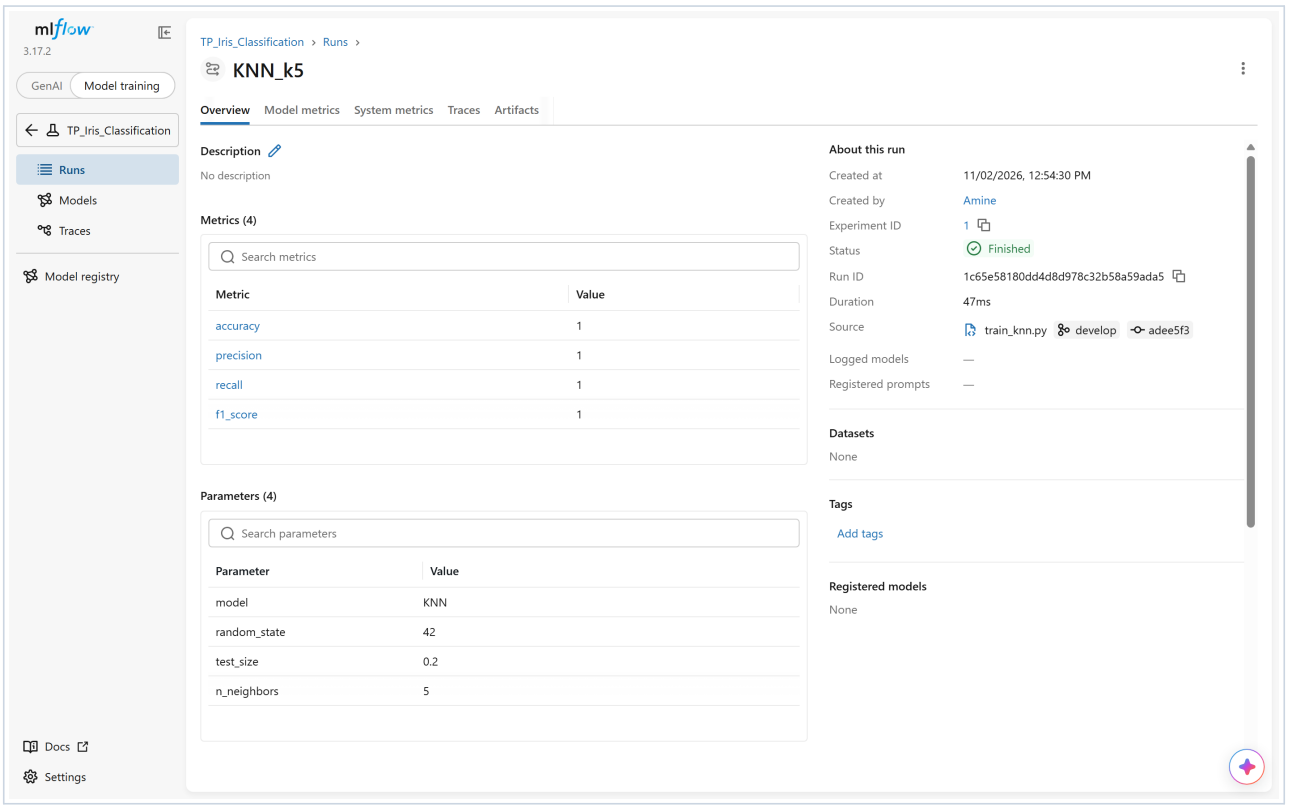



*Figure 9 — Fiche détaillée du run KNN_k5 (tag v3.0 : Accuracy = 1.0000, n_neighbors = 5, random_state = 42, test_size = 0.2).*

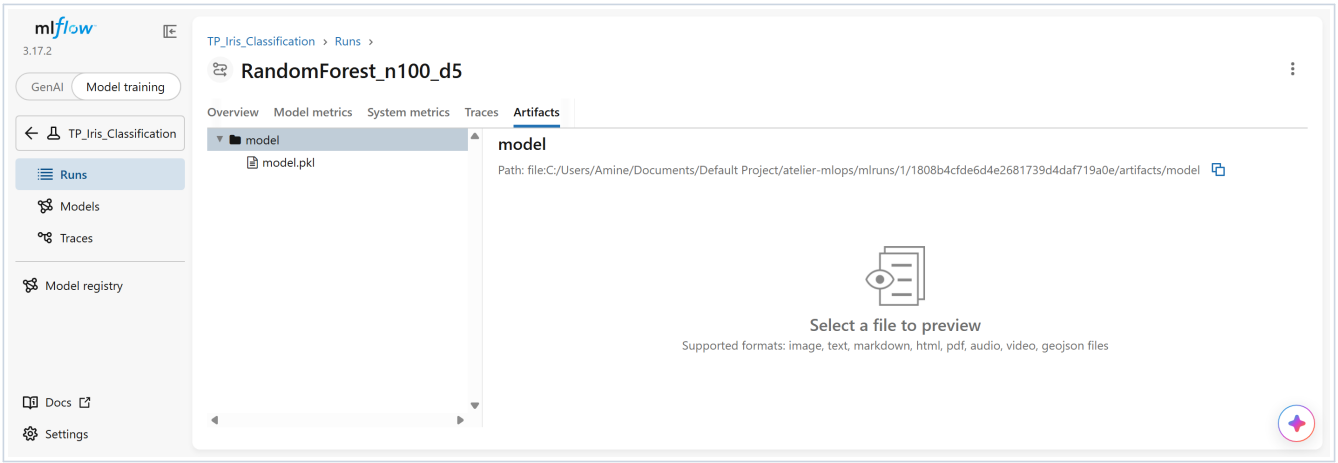



*Figure 10 — Onglet Artifacts dans MLflow UI : arborescence model/ dépliée confirmant l'enregistrement de l'artefact binaire model.pkl.*

## 5. TP 4 — Pipeline Intégré DVC, Reproductibilité et Extension CI/CD

Les fichiers params.yaml et dvc.yaml définissent de manière déclarative les deux stages du pipeline (train pour Random Forest et train_knn pour KNN), tandis que dvc.lock fige les hachages MD5 exacts de chaque entrée et sortie :

```
# params.yaml
train:
  n_estimators: 200
  max_depth: 8
  random_state: 42
  test_size: 0.2
knn:
  n_neighbors: 5

# dvc.yaml
stages:
  train:
    cmd: python src/train.py
    deps: [src/train.py, data/raw/iris.csv]
    params: [train.n_estimators, train.max_depth, train.random_state, train.test_size]
    outs: [models/model.pkl]
    metrics:
      - reports/metrics.json: {cache: false}
  train_knn:
    cmd: python src/train_knn.py
    deps: [src/train_knn.py, data/raw/iris.csv]
    params: [train.random_state, train.test_size, knn.n_neighbors]
    outs: [models/knn_model.pkl]
    metrics:
      - reports/metrics_knn.json: {cache: false}
```

**Restauration multi-versions, reproductibilité sur machine isolée et tests CI (Section 12) :** La restauration de v1.0 (git checkout v1.0 && dvc checkout) rétablit instantanément le modèle à 100 arbres (accuracy = 0.93333). En outre, un clone complet exécuté dans un répertoire vierge isolé hors du projet (sans dossier ../dvc-storage) télécharge automatiquement les 4 fichiers suivis via dvc pull (HTTPS), valide dvc repro et passe **4/4 tests Pytest** (qualité de iris.csv et wine.csv, cohérence pipeline, inférence des modèles et seuils d'acceptation accuracy & f1_score >= 0.85).




*Figure 11 — Exécution du pipeline intégré : graphe dvc dag, comparaison CLI v1.0 vs v2.0, démonstration de rollback vers v1.0 et validation Pytest (4 passed).*

## 6. Réponses aux Questions de Réflexion (Section 16)

### Q15. Quelle est la différence entre Git et DVC ?

Git est un système de contrôle de versions conçu pour le code source et les fichiers texte légers (.py, .yaml, .dvc, dvc.lock). DVC étend Git aux données volumineuses et aux modèles binaires en stockant leur contenu réel dans un cache adressé par empreinte MD5 (synchronisé avec un remote local ou cloud) tout en orchestrant l'exécution reproductible du pipeline.

### Q16. Pourquoi éviter de stocker un dataset volumineux directement dans Git ?

Chaque modification d'un fichier lourd est dupliquée dans l'historique .git de tous les collaborateurs, ce qui alourdit définitivement le dépôt, ralentit les opérations réseau (clone, fetch, push), dépasse les quotas des plateformes (limite de 100 Mo par fichier sur GitHub) et produit des diffs textuels inexploitables.

### Q17. Quelle est la différence entre dvc add et dvc repro ?

dvc add place un fichier statique existant (comme un dataset brut) sous suivi DVC en calculant son MD5 et en créant un fichier .dvc sans exécuter de traitement. dvc repro parcourt le graphe de dépendances défini dans dvc.yaml, compare les empreintes courantes des entrées et paramètres à dvc.lock, ne relance que les étapes modifiées et met à jour dvc.lock.

### Q18. Quel est le rôle du fichier dvc.lock ?

dvc.lock fige l'état exact résolu d'une exécution du pipeline : commandes lancées, valeurs effectives des hyperparamètres, tailles et hachages MD5 de toutes les dépendances (deps) et sorties (outs). Versionné dans Git, il associe de manière déterministe et vérifiable chaque commit à une version unique des données et des modèles.

### Q19. Comment MLflow permet-il de comparer deux expérimentations ?

Chaque exécution enregistrée dans le Backend Store (sqlite:///mlflow.db) conserve ses hyperparamètres, ses métriques et ses artefacts. La vue Compare de MLflow UI aligne côte à côte plusieurs runs en mettant en évidence les paramètres modifiés (Show diff only), compare les métriques et génère des graphiques interactifs (coordonnées parallèles, scatter/box plots), complétée par mlflow.search_runs().

### Q20. Comment retrouver les données et le modèle associés à un commit Git précis ?

En deux étapes : (1) exécuter git checkout <commit_ou_tag> (ex. git checkout v1.0) pour restaurer le code, params.yaml, dvc.lock et les fichiers .dvc de cet instant ; (2) exécuter dvc checkout (ou dvc pull si absent du cache local) afin de matérialiser exactement les datasets et le fichier modèle correspondants.

### Q21. Pourquoi la reproductibilité est-elle importante dans un projet d'intelligence artificielle ?

Un modèle d'IA dépend simultanément du code, des données, du prétraitement, des hyperparamètres et des graines aléatoires. La reproductibilité est indispensable pour valider scientifiquement l'impact d'une modification, garantir l'auditabilité et la traçabilité en production, effectuer un rollback fiable vers une version stable (v1.0) et collaborer sans écarts entre machines.

### Q22. Quelle est la différence entre le versioning des modèles avec DVC et leur suivi avec MLflow ?

DVC versionne le fichier physique du modèle (models/model.pkl) comme sortie déterministe du pipeline liée à un commit Git précis (orienté restauration et CI/CD). MLflow assure le suivi analytique de toutes les expérimentations (y compris exploratoires), la visualisation comparative des métriques et la gouvernance du cycle de vie via le Model Registry. Les deux outils sont donc complémentaires.

## 7. Synthèse de Vérification des Livrables (Section 17) et Conclusion

| Livrable attendu (Grille /20) | Preuve & Référence dans ce rapport | Statut |
|---|---|---|
| **Git & Repository GitHub (3 pts)** | github.com/anakin-skyguy/atelier-mlops — Figures 1 & 2 (main, develop, v1.0–v3.0) | **Complet** |
| **Versioning données DVC (4 pts)** | iris.csv.dvc, wine.csv.dvc, double remote & rollback MD5 — Figure 3 | **Complet** |
| **Tracking MLflow UI (4 pts)** | RF v1.0, RF v2.0 & KNN k=5 comparés + artefacts — Figures 4 à 10 | **Complet** |
| **Pipeline intégré DVC (4 pts)** | dvc.yaml, dvc.lock, params.yaml, dvc dag — Section 5 & Figure 11 | **Complet** |
| **Reproductibilité & Versions (3 pts)** | Rollback v1.0, clone vierge HTTPS, Pytest 4/4 & GitHub Actions — Figure 11 | **Complet** |
| **Documentation & Réflexion (2 pts)** | README complet + Réponses détaillées aux questions Q15 à Q22 — Section 6 | **Complet** |

**Conclusion :** L'articulation de **Git** (versioning du code et des pointeurs), **DVC** (versioning des données et modèles, orchestration déterministe du pipeline) et **MLflow** (tracking analytique et comparaison visuelle des runs) permet de transformer un prototype Scikit-learn en un workflow MLOps rigoureux, auditable et intégralement reproductible.